Lucrarea 2 – Implementarea algoritmilor paraleli. Condiții de paralelism

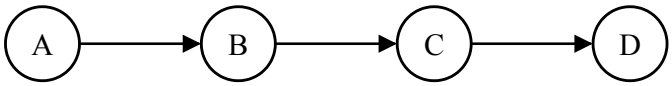
Laborator Algoritmi Paraleli și Distribuiți – rezolvarea exercițiilor

EXERCIȚIUL 1

Pentru o singură imagine avem task-urile: A – citirea căii fișierului, B – încărcarea imaginii, C – transformarea în nivele de gri, D – salvarea imaginii. Mulțimile de intrare (I) și ieșire (O):

- A: I_A = {sourceDirectory}, O_A = {fileName}
- B: I_B = {fileName}, O_B = {image}
- C: I_C = {image}, O_C = {image}
- D: I_D = {image, finalDirectory}, O_D = {fișierul salvat}

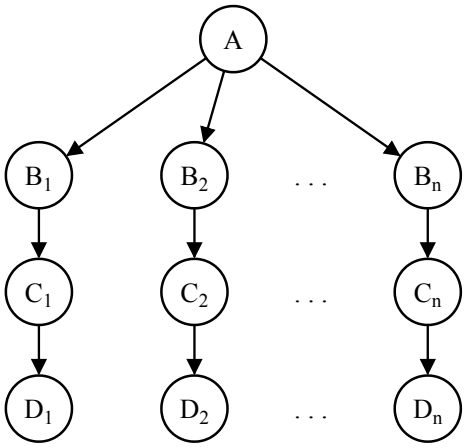
Graful dependențelor:



$O_A \cap I_B$ = {fileName} ≠ Φ, $O_B \cap I_C$ = {image} ≠ Φ, $O_C \cap I_D$ = {image} ≠ Φ, deci între task-uri există dependențe prin succesiune (A → B → C → D) și condițiile lui Bernstein nu sunt îndeplinite. Fiecare task are nevoie de rezultatul celui dinainte, așa că pentru o singură imagine problema **nu se poate paraleliza** la nivel de task-uri. Doar în interiorul lui C pixelii sunt independenți între ei, dar toți se află în același obiect Bitmap.

EXERCIȚIUL 2

Pentru un set de imagini, Task A (lista de fișiere) se execută o singură dată, apoi pentru fiecare imagine i se execută lanțul $B_i \rightarrow C_i \rightarrow D_i$:



Lanțurile pentru imagini diferite lucrează pe fișiere și obiecte Bitmap diferite, deci pentru i ≠ j: $I_i \cap O_j$ = Φ, $I_j \cap O_i$ = Φ, $O_i \cap O_j$ = Φ. Condițiile lui Bernstein sunt îndeplinite, așa că după Task A imaginile **se pot procesa în paralel**: fiecare thread preia o parte din imagini.

EXERCIȚIUL 3

Lista de imagini se împarte în N partiții consecutive. Threadul cu indexul `tid` procesează imaginile de la `tid·n/N` până la `(tid+1)·n/N − 1`, unde n este numărul de imagini:

```
//atentie la calea de pe disk unde se afla imaginile!!!
public static string sourceDirectory = @"C:\Studenti\SourceImages";
public static string finalDirectory = @"C:\Studenti\FinalImages";
public static string[] fileEntries; //lista partajata
public static int N;

public static void ThreadFunction(object parameter)
{
    int tid = (int)parameter;
    int start = tid * fileEntries.Length / N;
    int stop = (tid + 1) * fileEntries.Length / N;

    for (int i = start; i < stop; i++)
    {
        string fileName = fileEntries[i];
        using (Bitmap image = new Bitmap(fileName)) //Task B
        {
            for (int x = 0; x < image.Width; x++)  //Task C
            {
                for (int y = 0; y < image.Height; y++)
                {
                    Color pixelColor = image.GetPixel(x, y);
                    int gray = (pixelColor.R + pixelColor.G + pixelColor.B) / 3;
                    Color newColor = Color.FromArgb(gray, gray, gray);
                    image.SetPixel(x, y, newColor);
                }
            }
            image.Save(Path.Combine(finalDirectory, Path.GetFileName(fileName))); //Task D
        }
    }
}
```

Cu trei threaduri (varianta cu două threaduri este la fel, fără `th3` și cu `N = 2`):

```
static void Main(string[] args)
{
    fileEntries = Directory.GetFiles(sourceDirectory, "*.bmp").OrderBy(x => x).ToArray(); //Task A
    N = 3;

    Thread th1 = new Thread(ThreadFunction);
    Thread th2 = new Thread(ThreadFunction);
    Thread th3 = new Thread(ThreadFunction);
    th1.Start(0); //parametru - indexul partitiei
    th2.Start(1);
    th3.Start(2);
    th1.Join();
    th2.Join();
    th3.Join();
}
```

Cu N threaduri, unde N se dă ca parametru la rularea aplicației (în Visual Studio: Project → Properties → Debug → Command line arguments):

```
static void Main(string[] args)
{
    fileEntries = Directory.GetFiles(sourceDirectory, "*.bmp").OrderBy(x => x).ToArray(); //Task A
    N = int.Parse(args[0]);

    Thread[] threads = new Thread[N];
    for (int i = 0; i < N; i++)
    {
        threads[i] = new Thread(ThreadFunction);
        threads[i].Start(i); //parametru - indexul partitiei
    }
    for (int i = 0; i < N; i++)
    {
        threads[i].Join();
```

```
    }
}
```

EXERCIȚIUL 4 – TIMPUL DE EXECUȚIE

Am măsurat cu `Stopwatch` varianta serială (bucla `foreach` de la exercițiul 2) și varianta paralelă de la exercițiul 3, pentru N = 1 … 20:

```
Stopwatch timer = new Stopwatch();
timer.Start();
foreach (string fileName in fileEntries)
{
    ... //Task B, C, D - varianta seriala
}
timer.Stop();
Console.WriteLine("Serial: Elapsed = {0} (ms)", timer.Elapsed.TotalMilliseconds);

for (N = 1; N <= 20; N++)
{
    timer.Restart();
    ... //lansarea si asteptarea celor N threaduri, ca la exercitiul 3
    timer.Stop();
    Console.WriteLine("N = {0}: Elapsed = {1} (ms)", N, timer.Elapsed.TotalMilliseconds);
}
```

Măsurătorile le-am făcut pe macOS (8 procesoare logice). Pentru că System.Drawing nu este disponibil pe macOS, am folosit o variantă a aceluiași program în care imaginile sunt citite și salvate cu biblioteca SkiaSharp; algoritmul și împărțirea pe threaduri sunt aceleași. Rezultate pentru 40 de imagini de 500x500 px: T_{serial} = 318 ms, iar S = T_{serial} / $T_{paralel}$:

N	$T_{paralel}$ (ms)	S
1	327	0,97
2	202	1,57
3	137	2,32
4	148	2,15
5	148	2,15
6	128	2,48
7	134	2,37
8	136	2,34
9	134	2,37
10	138	2,30

N	$T_{paralel}$ (ms)	S
11	116	2,74
12	122	2,61
13	122	2,61
14	112	2,84
15	101	3,15
16	103	3,09
17	124	2,56
18	124	2,56
19	113	2,81
20	106	3,00

Timpul scade clar până la N = 3 (S ≈ 2,32). După aceea speedup-ul crește foarte puțin și oscilează între 2,1 și 3,2, deci nu ajunge la 8, numărul de procesoare logice. Peste 8 threaduri nu se mai câștigă aproape nimic, pentru că threadurile în plus doar își împart aceleași nuclee.

EXERCIȚIUL 4 – ALOCAREA DINAMICĂ A TASK-URILOR

Threadurile nu mai primesc o partiție fixă. Fiecare thread ia din listă următoarea imagine neprocesată, o marchează ca preluată (șir gol) și, după ce o termină, ia alta. Accesul la listă se face exclusiv, cu `lock`:

```
static object _lock = new object();

public static void ThreadFunction(object parameter)
{
    int tid = (int)parameter;
```

```
    while (true)
    {
        string fileName = string.Empty;
        lock (_lock)
        {
            for (int i = 0; i < fileEntries.Length && fileName == string.Empty; i++)
            {
                if (fileEntries[i] != string.Empty)
                {
                    fileName = fileEntries[i];
                    fileEntries[i] = string.Empty; //imaginea a fost preluata
                }
            }
        }
        if (fileName == string.Empty)
            break; //nu mai sunt imagini de procesat

        using (Bitmap image = new Bitmap(fileName)) //Task B
        {
            ... //Task C si Task D, ca la exercitiul 3
        }
    }
}
```

Comparație cu varianta statică de la exercițiul 3, pe 16 imagini de dimensiuni diferite (de la 2000x2000 la 100x100 px, cele mari la începutul listei). T_{serial} = 593 ms:

N	Static (ms)	Dinamic (ms)	S static	S dinamic
2	527	294	1,13	2,02
4	402	204	1,48	2,91
8	264	189	2,25	3,14

Varianta dinamică este mai rapidă pentru toate valorile lui N (de exemplu la N = 4: 204 ms față de 402 ms). La varianta statică primul thread primește imaginile mari de la începutul listei, iar celelalte termină repede și stau, pe când la varianta dinamică fiecare thread ia o imagine nouă imediat ce a terminat-o pe cea anterioară.

EXERCIȚIUL 5

Am testat variantele de la exercițiul 3 (statică) și exercițiul 4 (dinamică), cu N = 4, pe imagini mici (40 × 100x100 px), medii (40 × 500x500 px) și mari (8 × 2000x2000 px):

Imagini	Serial (ms)	Static N=4 (ms)	Dinamic N=4 (ms)	S static	S dinamic
Mici (40 × 100x100)	44	10	8	4,40	5,50
Medii (40 × 500x500)	307	125	121	2,46	2,54
Mari (8 × 2000x2000)	895	338	345	2,65	2,59

Pentru toate tipurile de imagini variantele paralele sunt de aproximativ 2,5–5,5 ori mai rapide decât cea serială. La imaginile mici timpii sunt de doar câteva milisecunde, așa că speedup-ul mare de acolo nu este foarte sigur. Când imaginile au aceeași dimensiune, varianta statică și cea dinamică dau timpi aproape egali (125/121 ms și 338/345 ms), pentru că partițiile sunt deja echilibrate.

Lucrarea 3 – Implementarea algoritmilor paraleli. Accesul concurent la resurse

Laborator Algoritmi Paraleli și Distribuiți – rezolvarea exercițiilor

EXERCIȚIUL 1

Am implementat și rulat codul din laborator (proiect Console Application, referință la `System.Drawing`):

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading;
using System.Drawing;
using System.IO;
using System.Diagnostics;

namespace L2
{
    class Program
    {
        static string sourceDirectory = @"C:\Studenti\SourceImages";
        static string finalDirectory = @"C:\Studenti\FinalImages";
        static string[] fileEntries;
        static int N = 4;

        static object _lock = new object();

        static void Main(string[] args)
        {
            fileEntries = Directory.GetFiles(sourceDirectory, "*.bmp").OrderBy(x => x).ToArray();
//Task A

            Thread[] childThreads = new Thread[N];
            for (int i = 0; i < N; i++)
            {
                childThreads[i] = new Thread(ThreadFunction);
                childThreads[i].Start(i);
            }
            for (int i = 0; i < N; i++)
            {
                childThreads[i].Join();
            }

            Console.ReadKey();
        }

        public static void ThreadFunction(object parameter)
        {
            int tid = (int)parameter; //thread ID
            bool exista = true;
            string fileName = string.Empty;
            while (exista)
            {
                exista = false;
                lock (_lock) //enter critical zone
                {
                    for (int i = 0; i < fileEntries.Length && !exista; i++)
                    {
                        if (!string.IsNullOrEmpty(fileEntries[i]))
                        {
                            exista = true;
                            fileName = fileEntries[i];
                            fileEntries[i] = string.Empty; //mark file as processed (by convention
an empty file == file is already taken)
                            Console.WriteLine("Thread ID {0} take image {1} ", tid, i);
                        }
```

```
                    }
                }
                if (exista)
                {
                    using (Bitmap image = new Bitmap(fileName)) //Task B
                    {
                        for (int x = 0; x < image.Width; x++)  //Task C
                        {
                            for (int y = 0; y < image.Height; y++)
                            {
                                Color pixelColor = image.GetPixel(x, y);
                                int gray = (pixelColor.R + pixelColor.G + pixelColor.B) / 3;
                                Color newColor = Color.FromArgb(gray, gray, gray);
                                image.SetPixel(x, y, newColor);
                            }
                        }
                        image.Save(Path.Combine(finalDirectory, Path.GetFileName(fileName)));
//Task D
                    }
                }
            }
        }
    }
}
```

Am testat cu 16 imagini de aceeași dimensiune (800x800 px) și cu 16 imagini de dimensiuni diferite (între 100x100 și 2000x2000 px), pentru N = 1, 2 și 4.

Exemplu de rulare, imagini de dimensiuni diferite, N = 4:

```
Thread ID 0 take image 0                        Thread ID 3 take image 8
Thread ID 2 take image 1                        Thread ID 3 take image 9
Thread ID 1 take image 2                        Thread ID 1 take image 10
Thread ID 3 take image 3                        Thread ID 3 take image 11
Thread ID 2 take image 4                        Thread ID 3 take image 12
Thread ID 3 take image 5                        Thread ID 3 take image 13
Thread ID 3 take image 6                        Thread ID 2 take image 14
Thread ID 3 take image 7                        Thread ID 2 take image 15
```

Observații:

- **N = 1:** un singur thread preia toate imaginile, pe rând, în ordinea din listă (0, 1, 2, …, 15).
- **Imagini de aceeași dimensiune:** imaginile sunt preluate tot în ordinea din listă, iar threadurile le iau aproximativ pe rând. Fiecare thread procesează același număr de imagini (N = 2: 8 + 8, N = 4: câte 4).
- **Imagini de dimensiuni diferite:** ordinea de preluare rămâne cea din listă, dar threadurile nu mai procesează același număr de imagini (la N = 4: 9, 4, 2 și 1 imagini). Threadul care a primit o imagine mare rămâne ocupat mai mult, iar threadurile libere preiau imaginile următoare. Astfel taskurile se împart dinamic, după cât de repede termină fiecare thread.

EXERCIȚIUL 2

În varianta de la exercițiul 1, la fiecare preluare threadul parcurge lista de la început în zona critică, iar `Console.WriteLine` (o operație lentă) se execută tot în zona critică. Propunere: păstrăm un index `next` al următoarei imagini nepreluate. În zona critică doar citim și incrementăm indexul, iar restul operațiilor se fac în afara ei:

```
        static int next = 0;
        ...

        public static void ThreadFunction(object parameter)
        {
            int tid = (int)parameter; //thread ID
            bool exista = true;
            string fileName = string.Empty;
```

```
            while (exista)
            {
                int i;
                lock (_lock) //enter critical zone
                {
                    i = next;
                    next++;
                } //exit critical zone

                exista = i < fileEntries.Length;
                if (exista)
                {
                    fileName = fileEntries[i];
                    Console.WriteLine("Thread ID {0} take image {1} ", tid, i);

                    using (Bitmap image = new Bitmap(fileName)) //Task B
                    {
                        ... //Task C si Task D neschimbate
                    }
                }
            }
        }
```

Zona critică are acum doar două instrucțiuni, de durată constantă, indiferent de numărul de imagini, așa că threadurile stau mult mai puțin blocate în așteptarea lock-ului.

EXERCIȚIUL 3

Un lock pentru fiecare task. Vectorul `_locks` are aceeași lungime ca lista de imagini, iar threadul blochează doar elementul pe care îl verifică:

```
        //static object _lock = new object();
        static object[] _locks;

        static void Main(string[] args)
        {
            fileEntries = Directory.GetFiles(sourceDirectory, "*.bmp").OrderBy(x => x).ToArray();
//Task A

            _locks = new object[fileEntries.Length];
            for (int i = 0; i < _locks.Length; i++)
                _locks[i] = new object();

            ... //lansarea si asteptarea celor N threaduri, ca la exercitiul 1
        }

        public static void ThreadFunction(object parameter)
        {
            int tid = (int)parameter; //thread ID
            bool exista = true;
            string fileName = string.Empty;
            while (exista)
            {
                exista = false;

                for (int i = 0; i < fileEntries.Length && !exista; i++)
                {
                    lock (_locks[i]) //enter critical zone at task level
                    {
                        if (!string.IsNullOrEmpty(fileEntries[i]))
                        {
                            exista = true;
                            fileName = fileEntries[i];
                            fileEntries[i] = string.Empty; //mark file as processed (by convention
an empty file == file is already taken)
                            Console.WriteLine("Thread ID {0} take image {1} ", tid, i);
                        }
                    }
                }

                if (exista)
```

```
                {
                    ... //Task B, C, D neschimbate
                }
            }
        }
```

Accesul la listă nu mai este serializat: două threaduri pot verifica în același timp elemente diferite din listă și se blochează doar când încearcă să ia aceeași imagine.

EXERCIȚIUL 4

Două liste de taskuri, fiecare cu jumătate din fișierele din folder, și două variabile de tip lock. Threadurile cu ID par lucrează pe lista 1, iar cele cu ID impar pe lista 2:

```
        static string[] fileEntries1;
        static string[] fileEntries2;
        static int N = 4;

        static object _lock1 = new object();
        static object _lock2 = new object();

        static void Main(string[] args)
        {
            string[] fileEntries = Directory.GetFiles(sourceDirectory, "*.bmp").OrderBy(x =>
x).ToArray(); //Task A
            fileEntries1 = fileEntries.Take(fileEntries.Length / 2).ToArray();
            fileEntries2 = fileEntries.Skip(fileEntries.Length / 2).ToArray();

            ... //lansarea si asteptarea celor N threaduri, ca la exercitiul 1
        }

        public static void ThreadFunction(object parameter)
        {
            int tid = (int)parameter; //thread ID
            string[] fileEntries = (tid % 2 == 0) ? fileEntries1 : fileEntries2;
            object _lock = (tid % 2 == 0) ? _lock1 : _lock2;

            bool exista = true;
            string fileName = string.Empty;
            while (exista)
            {
                exista = false;
                lock (_lock) //enter critical zone
                {
                    for (int i = 0; i < fileEntries.Length && !exista; i++)
                    {
                        if (!string.IsNullOrEmpty(fileEntries[i]))
                        {
                            exista = true;
                            fileName = fileEntries[i];
                            fileEntries[i] = string.Empty; //mark file as processed (by convention
an empty file == file is already taken)
                            Console.WriteLine("Thread ID {0} (lista {1}) take image {2} ", tid, tid
% 2 + 1, i);
                        }
                    }
                }
                if (exista)
                {
                    ... //Task B, C, D neschimbate
                }
            }
        }
```

Fiecare listă are lock-ul ei, deci threadurile se blochează doar între ele pe aceeași listă (jumătate din threaduri pe fiecare lock), nu toate pe aceeași resursă. Soluția este una intermediară între un singur lock (exercițiul 1) și câte un lock pentru fiecare task (exercițiul 3).